

# Progetto CROSS

## (exChange order bookS Service)

Autrice: Marta Menna

---

L'implementazione del progetto **CROSS (exChange order bookS Service)** sviluppato in Java 8, fa uso di threadpool, selector e I/O non bloccante (NIO) per ottimizzare la gestione di più connessioni.

### Architettura del progetto

L'architettura del sistema è di tipo client-server, il server gestisce la logica di business e la persistenza dei dati (utente e ordini) e il client interagisce col server attraverso diverse modalità di comunicazione (TCP, UDP, RMI).

### Componenti principali del **Server**:

#### ❖ **Modulo di Comunicazione principale (TCP/NIO):**

**ServerMain.java** avvia il server che utilizza Java NIO per gestire le connessioni TCP non bloccanti. Il **SelectorManager.java** si occupa della selezione degli eventi di I/O (accettazione di nuove connessioni, lettura/scrittura sui canali) e delega la gestione delle singole connessioni ad un **ThreadPoolExecutor** che esegue la task **ConnectionHandler.java** (connessione 1 a 1 col client). Questo approccio massimizza l'efficienza nella gestione delle connessioni simultanee con più client.

#### ❖ **Gestione Utenti (RMI e Persistenza):**

Lo **UserManager.java** è responsabile della registrazione, autenticazione (login/logout), gestione delle sessioni degli utenti (implementa un meccanismo di disconnessione per inattività) e persistenza dei dati utente.

La registrazione ( **RegistrationServiceImpl.java** ) avviene tramite RMI, mentre login, updateCredentials e logout sono gestiti via TCP. I dati degli utenti vengono salvati sul file JSON **data/users.json**.

#### ❖ **Gestione Order Book:**

**OrderBookManager.java** è il cuore del sistema. Gestisce gli ordini di acquisto ( **bidLimitOrders**, **bidStopOrders** ) e di vendita ( **askLimitOrders**, **askStopOrders** )

utilizzando una `Concurrent SkipListMap` che permette di mantenere gli ordini ordinati per prezzo e garantire l'accesso thread-safe.

Il matching degli ordini (Limit e Stop) è implementato da due diversi `Scheduler`:

- `limitOrdersScheduler` esegue periodicamente l'algoritmo di matching dei `Limit orders`
- `stopOrdersScheduler` trasforma in `Market orders` gli `Stop orders` che superano la soglia.

Gli ordini completati vengono salvati sul file JSON `data/storicoOrdini.json`.

#### ❖ Sistema di Notifica (UDP Multicast e UDP):

`NotificationService.java` permette di inviare ai client notifiche push in tempo reale (il superamento di una soglia di prezzo) tramite un gruppo multicast UDP, grazie ad uno scheduler interno che controlla periodicamente le variazioni di prezzo; questo si attiva una volta che viene istanziata la classe (in questo caso nel main).

`UDPNotificationSender.java` permette di inviare notifiche udp asincrone agli utenti a cui appartengono uno o più ordini che sono stati completati, interagisce con `UserManager.java` e `OrderBookManager.java` e mantiene una `Concurrent HashMap` che associa l'`orderId` all'`utente`.

### Componenti principali del Client:

#### ❖ Interfaccia Utente (CLI):

Il `ClientMain.java` avvia l'applicazione client, che interagisce col server da linea di comando.

#### ❖ Servizio RMI per Registrazione:

Il `RegistrationClient.java` si connette al servizio RMI offerto dal server per le operazioni di registrazione di nuovi utenti.

#### ❖ Comunicazione Server (TCP/NIO):

Il `ConnectionHandlerclien.java` gestisce la comunicazione TCP con il server, inviando richieste in formato JSON e interpretando le risposte.

#### ❖ Ricezione Notifiche (UDP Multicast e UDP):

`MulticastNotificationReceiver.java` permette al client di unirsi al gruppo multicast e di ricevere le notifiche di prezzo dal server, visualizzandole solo se pertinenti alla soglia

```
C:\Users\994565\.jdk\corretto-1.8.0_452\bin\java.exe ...
> Benvenuto su CROSS!
[UDP Receiver] Avviato sulla porta 12347.

Scegli un'opzione:
1 Register
2 Login
3 Quit
1
Register
Username:
> antonella_clerici

Password:
> 123456

102 Username non disponibile
```

impostata dall'utente. Mentre `UDPNotificationReceiver.java` rimane in ascolto di notifiche riguardanti l'esecuzione di un ordine precedentemente sottomesso dallo user.

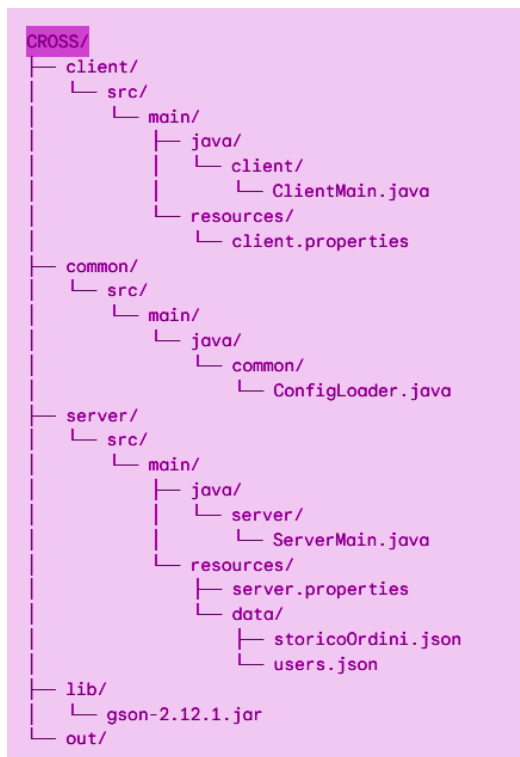
## Gestione della Concorrenza

Per garantire la coerenza dei dati in un ambiente concorrente, ho adottato le seguenti soluzioni:

- **Strutture dati Thread-Safe:** ho utilizzato `Concurrent HashMap` e `Concurrent SkipListMap` per la gestione di utenti e ordini ( strutture dati condivise ) di modo da garantire l'accesso sicuro e concorrente.
- **Sincronizzazione:** Blocchi `synchronized` sono presenti nei metodi 'critici' (appartenenti a `OrderBookManager` e `UserManager`) dove sono necessarie operazioni atomiche ad esempio nel caso di scrittura sulle strutture dati condivise.
- **Task Scheduling:** `Scheduled ExecutorService` è utilizzato per eseguire task periodici, come il matching degli ordini, la persistenza dei dati o l'invio di notifiche push, garantendo che queste operazioni vengano eseguite in modo asincrono e non blocchino il flusso principale del server.

## Istruzioni per l'Esecuzione di CROSS:

Struttura del progetto e posizionamento file:



(il codice è già stato compilato ma per completezza è presente la guida su compilazione ed esecuzione)

Per **COMPILARE** i files sorgente Java, posizionati nella directory radice del progetto (`CROSS/`) ed esegui i seguenti passaggi:

1. Compila ciascun modulo separatamente, indirizzando l'output alla directory `out/`.

Per Linux/macOS

```
javac -d out -cp "lib/gson-2.12.1.jar" client/src/main/java/client/*.java
javac -d out -cp "lib/gson-2.12.1.jar" common/src/main/java/common/*.java
javac -d out -cp "lib/gson-2.12.1.jar" server/src/main/java/server/*.java
```

Per Windows

```
javac -d out -cp "lib\gson-2.12.1.jar" client\src\main\java\client\*.java
javac -d out -cp "lib\gson-2.12.1.jar" common\src\main\java\common\*.java
javac -d out -cp "lib\gson-2.12.1.jar" server\src\main\java\server\*.java
```

2. Copia dei file risorse nella directory di output per renderli accessibili al runtime.

Per Windows `copy`

```
copy client\src\main\resources\client.properties out\
copy server\src\main\resources\server.properties out\
copy server\src\main\resources\data\storicoOrdini.json out\data\
copy server\src\main\resources\data\users.json out\data\
```

Dopo questi passaggi, la directory `out/` dovrebbe contenere sia i file `.class` compilati che i file `.properties` e `.json`, posizionati correttamente per essere trovati dal `ConfigLoader`.

## Esecuzione del Server

Dopo la compilazione e la copia delle risorse, esegui il server dalla directory radice del progetto (`CROSS/`). Il classpath deve includere sia la directory `out/` (dove si trovano i file `.class` e le risorse) sia il file JAR di Gson.

Per Windows: `java -cp "lib\gson-2.12.1.jar;out" server.ServerMain`

Il server stamperà messaggi sulla console indicando l'avvio del servizio RMI e l'accettazione di nuove connessioni.

```
C:\Users\994565\.jdk\corretto-1.8.0_452\jre\lib\rt.jar;C:\Users\994565\Desktop\CROSS\out\production\server;C:\Users\994565\Desktop\CROSS\lib\gson-2.12.1.jar;
C:\Users\994565\Desktop\CROSS\out\production\common server.ServerMain
Server avviato sulla porta 12345
> 11 utenti caricati con successo da: C:\Users\994565\Desktop\CROSS\out\production\server\data\users.json
> Caricati 42020 ordini da: C:\Users\994565\Desktop\CROSS\out\production\server\data\storicoOrdini.json
Servizio RMI avviato sulla porta 1099
SelectorManager avviato
Nuova connessione accettata: /127.0.0.1:51585
java.nio.channels.SocketChannel[connected local=/127.0.0.1:12345 remote=/127.0.0.1:51585] Messaggio ricevuto: {"operation":"Login","values":{"password":"123456",
"threshold":1000,"username":"antonella_clerici"}}}
```

## Esecuzione del Client

Una volta che il server è in esecuzione, può essere avviato il client.

Per Windows: `java -cp "lib\gson-2.12.1.jar;out" client.ClientMain`

Il client presenterà un menu interattivo sulla console per interagire con il sistema.

```
C:\Users\994565\.jdk\corretto-1.8.0_452\bin\java.exe ...
> Benvenuto su CROSS!
[UDP Receiver] Avviato sulla porta 12347.

Scegli un'opzione:
1 Register
2 Login
3 Quit
> 1
Register
Username:
> antonella_clerici

Password:
> 123456

102 Username non disponibile

Scegli un'opzione:
1 Register
2 Login
3 Quit
> 2
Login
Username:
> antonella_clerici

Password:
> 123456

Threshold:
> 1000

> Messaggio inviato:
{"operation":"Login","values":{"password":"123456","threshold":1000,"username":"antonella_clerici"}}
response: 100.0
message: OK
```

```
Scegli un'opzione:
1 Update Credentials
2 Insert Limit Order
3 Insert Market Order
4 Insert Stop Order
5 Cancel Order
6 Get Price History
7 Logout
8 Quit
> 4
Insert Stop Order
Type:
> ask

size:
> 1000

price:
> 700

> Messaggio inviato:
{"operation":"InsertStopOrder","values":{"size":1000,"price":700,"type":"ask"}}
orderId: 1.0
```

## Esecuzione dei JAR da Riga di Comando

Naviga nella directory in cui si trovano i JAR `CROSS/out/artifacts`

Per Windows `java -jar server_executable.jar`  
`java -jar client_executable.jar`